SQL

SQL

- SQL is a Structured Query Language which is a computer language for storing, manipulating and retrieving data stored in relational database.
- SQL is the standard language for Relation Database System.

SQL Commands

- DDL – Data Definition Language
- DML – Data Manipulation Language
- DCL – Data Control Language
- DQL – Data Query Language

SQL Commands

DDL – Data Definition Language

Command	Description
CREATE	Creates a new table, a view of a table, or other object in the database.
ALTER	Modifies an existing database object, such as a table.
DROP	Deletes an entire table, a view of a table or other objects in the database.

SQL Commands

DML – Data Manipulation Language

Command	Description
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Delete records

SQL Commands

DCL – Data Control Language

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from users

DQL – Data Query Language

Command	Description
SELECT	Retrieves certain records from one or more tables

CREATE TABLE

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
   column3 datatype,
   .....
   columnN datatype,
   PRIMARY KEY( one or more columns )
);
```

```
CREATE TABLE CUSTOMERS (
   ID   INT                    NOT NULL,
   NAME VARCHAR (20)           NOT NULL,
   AGE  INT                    NOT NULL,
   ADDRESS  CHAR (25) ,
   SALARY   DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

DROP DATABASE / TABLE

```
DROP DATABASE DatabaseName;
DROP TABLE table_name;
```

```
DROP DATABASE dbEnrollment;
DROP TABLE tblStudent;
```

SELECT QUERY

```
SELECT column1, column2, columnN FROM table_name;
```

- column1, column2... are the fields of a table whose values you want to fetch

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT ID, NAME,
SALARY FROM
CUSTOMERS;
```

ID	NAME	SALARY
1	Ramesh	2000.00
2	Khilan	1500.00
3	kaushik	2000.00
4	Chaitali	6500.00
5	Hardik	8500.00
6	Komal	4500.00
7	Muffy	10000.00

SELECT QUERY

```
SELECT * FROM table_name;
```

- column1, column2... are the fields of a table whose values you want to fetch

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT * FROM
CUSTOMERS;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

SELECT with WHERE clause

```
SELECT column1, column2, columnN FROM table_name
WHERE [condition]
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000;
```

ID	NAME	SALARY
4	Chaitali	6500.00
5	Hardik	8500.00
6	Komal	4500.00
7	Muffy	10000.00

WHERE clause with AND operator

```
SELECT column1, column2, columnN FROM table_name
WHERE [condition1] AND [condition2]...AND [conditionN];
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000 AND age < 25;
```

ID	NAME	SALARY
6	Komal	4500.00
7	Muffy	10000.00

WHERE clause with OR operator

```
SELECT column1, column2, columnN FROM table_name
WHERE [condition1] OR [condition2]...OR[conditionN];
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000 OR age < 25;
```

ID	NAME	SALARY
3	kaushik	2000.00
4	Chaitali	6500.00
5	Hardik	8500.00
6	Komal	4500.00
7	Muffy	10000.00

UPDATE

```
UPDATE table_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
UPDATE CUSTOMERS
SET ADDRESS = 'Pune'
WHERE ID = 6;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Pune	4500.00
7	Muffy	24	Indore	10000.00

LIKE

- It is used to compare a value to similar values using wildcard operators:
  - percent sign (%) - represents zero, one or multiple characters
  - underscore (_) - represents a single number or character

Statement	Description
WHERE SALARY LIKE '200%'	Finds any values that start with 200.
WHERE SALARY LIKE '%200%'	Finds any values that have 200 in any position.
WHERE SALARY LIKE '_00%'	Finds any values that have 00 in the second and third positions.
WHERE SALARY LIKE '2_%_%'	Finds any values that start with 2 and are at least 3 characters in length.
WHERE SALARY LIKE '%2'	Finds any values that end with 2.
WHERE SALARY LIKE '_2%3'	Finds any values that have a 2 in the second position and end with a 3.
WHERE SALARY LIKE '2_ _ _3	Finds any values in a five-digit number that start with 2 and end with 3.

LIMIT

```
SELECT * FROM table_name LIMIT n
```

- to fetch limited number of records

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT * FROM
CUSTOMERS
LIMIT 3;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00

ORDER BY

```
SELECT column-list FROM table_name [WHERE condition]
[ORDER BY column1, column2, .. columnN] [ASC | DESC]
```

- used to sort the data in ascending or descending order, based on one or more columns

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT * FROM CUSTOMERS
ORDER BY NAME DESC;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
7	Muffy	24	Indore	10000.00
6	Komal	22	MP	4500.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
5	Hardik	27	Bhopal	8500.00
4	Chaitali	25	Mumbai	6500.00

GROUP BY

```
SELECT column1, column2 FROM table_name
WHERE [ conditions ]
GROUP BY column1, column2
ORDER BY column1, column2
```

- is used in collaboration with the SELECT statement to arrange identical data into groups

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Ramesh	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	kaushik	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT NAME, SUM(SALARY)
FROM CUSTOMERS
GROUP BY NAME;
```

NAME	SUM(SALARY)
Hardik	8500.00
kaushik	8500.00
Komal	4500.00
Muffy	10000.00
Ramesh	3500.00

DISTINCT

```
SELECT DISTINCT column1, column2,.....columnN
FROM table_name
WHERE [condition]
```

- is used in conjunction with the SELECT statement to eliminate all the duplicate records and fetching only unique records

```
+----------+
| SALARY   |
+----------+
|  1500.00 |
|  2000.00 |
|  2000.00 |
|  4500.00 |
|  6500.00 |
|  8500.00 |
| 10000.00 |
+----------+
```

```
SELECT DISTINCT SALARY FROM CUSTOMERS
```

```
+----------+
| SALARY   |
+----------+
|  1500.00 |
|  2000.00 |
|  4500.00 |
|  6500.00 |
|  8500.00 |
| 10000.00 |
+----------+
```

COUNT(), AVG() and SUM() Functions

- The COUNT() function returns the number of rows that matches a specified criterion.

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

- The AVG() function returns the average value of a numeric column.

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

- The SUM() function returns the total sum of a numeric column.

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

SQL Constraints

- Constraints are the rules enforced on the data columns of a table.
- These are used to limit the type of data that can go into a table.
  - This ensures the accuracy and reliability of the data in the database.
- Constraints could be either on a column level or a table level.
  - column level constraints are applied only to one column
  - table level constraints are applied to the whole table

SQL Constraints

- NOT NULL Constraint − Ensures that a column cannot have NULL value.
- DEFAULT Constraint − Provides a default value for a column when none is specified.
- UNIQUE Constraint − Ensures that all values in a column are different.
- PRIMARY Key − Uniquely identifies each row/record in a database table.
- FOREIGN Key − Uniquely identifies a row/record in any of the given database table.
- CHECK Constraint − The CHECK constraint ensures that all the values in a column satisfies certain conditions.
- INDEX − Used to create and retrieve data from the database very quickly.

Dropping Constraints

- Any constraint that you have defined can be dropped using the ALTER TABLE command with the DROP CONSTRAINT option.
- Ex.

```
ALTER TABLE EMPLOYEES DROP CONSTRAINT EMPLOYEES_PK;
```

  - to drop the primary key constraint in the EMPLOYEES table

Integrity Constraints

- Integrity constraints are used to ensure accuracy and consistency of the data in a relational database.
- Data integrity is handled in a relational database through the concept of referential integrity.
- There are many types of integrity constraints that play a role in Referential Integrity
  - Primary Key, Foreign Key, Unique Constraints etc.

SQL JOINS

SQL Joins clause is used to combine records from two or more tables in a database

- INNER JOIN – returns rows when there is a match in both tables.
- LEFT JOIN – returns all rows from the left table, even if there are no matches in the right table.
- RIGHT JOIN – returns all rows from the right table, even if there are no matches in the left table.
- FULL JOIN – returns rows when there is a match in one of the tables.
- SELF JOIN – is used to join a table to itself as if the table were two tables, temporarily renaming at least one table in the SQL statement.
- CARTESIAN JOIN – returns the Cartesian product of the sets of records from the two or more joined tables.

UNION

- SQL UNION clause/operator is used to combine the results of two or more SELECT statements without returning any duplicate rows.
- To use this UNION clause, each SELECT statement must have:
  - The same number of columns selected
  - The same number of column expressions
  - The same data type and
  - Have them in the same order

  But they need not have to be in the same length.

UNION

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]

UNION

SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

UNION

Table 1 – CUSTOMERS Table is as follows.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT ID, NAME, AMOUNT, DATE FROM
CUSTOMERS LEFT JOIN ORDERS ON
CUSTOMERS.ID = ORDERS.CUSTOMER_ID

UNION

SELECT ID, NAME, AMOUNT, DATE FROM
CUSTOMERS RIGHT JOIN ORDERS ON
CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Table 2 – ORDERS Table is as follows.

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

ID	NAME	AMOUNT	DATE
1	Ramesh	NULL	NULL
2	Khilan	1560	2009-11-20 00:00:00
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00
5	Hardik	NULL	NULL
6	Komal	NULL	NULL
7	Muffy	NULL	NULL

INTERSECT

- This is used to combine two SELECT statements, but returns rows only from the first SELECT statement that are identical to a row in the second SELECT statement.

EXCEPT

- This combines two SELECT statements and returns rows from the first SELECT statement that are not returned by the second SELECT statement.

NULL Values

- The term used to represent a missing value.
- A NULL value in a table is a value in a field that appears to be blank.
- NULL value is different than a zero value or a field that contains spaces

NULL Values

```
CREATE TABLE CUSTOMERS(
    ID INT              NOT NULL,
    NAME VARCHAR (20)   NOT NULL,
    AGE INT             NOT NULL,
    ADDRESS CHAR (25),
    SALARY DECIMAL (18, 2),
    PRIMARY KEY (ID)
);
```

- NOT NULL signifies that column should always accept an explicit value of the given data type
- two columns which did not use NOT NULL, means these columns could be NULL

NULL Values

- Use the **IS NULL** or **IS NOT NULL** operators to check for a NULL value

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	
7	Muffy	24	Indore	

```
SELECT * FROM CUSTOMERS
WHERE SALARY IS NOT NULL;
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00

ALIAS

- A table or a column can be renamed temporarily by giving another name known as Alias.
- The renaming is a temporary change and the actual table name does not change in the database.

```
SELECT column1, column2....
FROM table_name AS alias_name
WHERE [condition];
```


```
SELECT column1, column2....
FROM table_name AS alias_name
WHERE [condition];
```

ALTER TABLE

- SQL ALTER TABLE command is used to add, delete or modify columns in an existing table.
- It is also used to add and drop various constraints on an existing table..

ALTER TABLE

Command	Description
ALTER TABLE table_name ADD column_name datatype;	add a New Column in an existing table
ALTER TABLE table_name DROP COLUMN column_name;	DROP COLUMN in an existing table
ALTER TABLE table_name MODIFY COLUMN column_name datatype;	change the DATA TYPE of a column in a table
ALTER TABLE table_name MODIFY column_name datatype NOT NULL;	add a NOT NULL constraint to a column in a table
ALTER TABLE table_name DROP CONSTRAINT MyPrimaryKey;	DROP PRIMARY KEY constraint from a table

ALTER TABLE

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
ALTER TABLE CUSTOMERS ADD SEX char(1);
```

ID	NAME	AGE	ADDRESS	SALARY	SEX
1	Ramesh	32	Ahmedabad	2000.00	NULL
2	Ramesh	25	Delhi	1500.00	NULL
3	kaushik	23	Kota	2000.00	NULL
4	kaushik	25	Mumbai	6500.00	NULL
5	Hardik	27	Bhopal	8500.00	NULL
6	Komal	22	MP	4500.00	NULL
7	Muffy	24	Indore	10000.00	NULL

HAVING

- **HAVING** clause enables you to specify conditions that filter which group results appear in the results.
- The **WHERE** clause places conditions on the selected columns, whereas the **HAVING** clause places conditions on groups created by the **GROUP BY** clause.
- The **HAVING** clause must follow the **GROUP BY** clause in a query and must also precede the **ORDER BY** clause if used.

```
SELECT column1, column2
FROM table1, table2
WHERE [ conditions ]
GROUP BY column1, column2
HAVING [ conditions ]
ORDER BY column1, column2
```

HAVING

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT * FROM CUSTOMERS
GROUP BY age
HAVING COUNT(age) >= 2;
```

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00

SUBQUERY

- A Subquery or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause.
- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.

SUBQUERY

There are a few rules that subqueries must follow

- Subqueries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.
- Subqueries that return more than one row can only be used with multiple value operators such as the IN operator.
- The BETWEEN operator cannot be used with a subquery. However, the BETWEEN operator can be used within the subquery.

Subqueries with the SELECT Statement

```
SELECT column_name [, column_name ]
FROM   table1 [, table2 ]
WHERE  column_name OPERATOR
    (SELECT column_name [, column_name ]
    FROM table1 [, table2 ]
    [WHERE])
```

Subqueries with the SELECT Statement

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

```
SELECT * FROM CUSTOMERS
WHERE ID IN (
    SELECT ID FROM CUSTOMERS
    WHERE SALARY > 4500) ;
```

ID	NAME	AGE	ADDRESS	SALARY
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
7	Muffy	24	Indore	10000.00

Subqueries with the INSERT Statement

- Subqueries also can be used with INSERT statements. The INSERT statement uses the data returned from the subquery to insert into another table.
-
```
INSERT INTO table_name [ (column1 [, column2 ]) ]
    SELECT [ *|column1 [, column2 ]
    FROM table1 [, table2 ]
    [ WHERE VALUE OPERATOR ]
```

Subqueries with the INSERT Statement

- Example: Consider a table CUSTOMERS_BKP with similar structure as CUSTOMERS table. Now to copy the complete CUSTOMERS table into the CUSTOMERS_BKP table, you can use the following syntax.

```
INSERT INTO CUSTOMERS_BKP
   SELECT * FROM CUSTOMERS
   WHERE ID IN (SELECT ID
   FROM CUSTOMERS) ;
```

Subqueries with the UPDATE Statement

- The subquery can be used in conjunction with the UPDATE statement. Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
   (SELECT COLUMN_NAME
   FROM TABLE_NAME)
   [ WHERE) ]
```

Subqueries with the UPDATE Statement

- Example: Assuming, we have CUSTOMERS_BKP table available which is backup of CUSTOMERS table. The following example updates SALARY by 0.25 times in the CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

```
UPDATE CUSTOMERS
 SET SALARY = SALARY * 0.25
 WHERE AGE IN
    (SELECT AGE FROM CUSTOMERS_BKP
   WHERE AGE >= 27 );
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	125.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	2125.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Subqueries with the DELETE Statement

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
    (SELECT COLUMN_NAME
    FROM TABLE_NAME)
    [ WHERE) ]
```

Subqueries with the DELETE Statement

- Example: Assuming, we have a CUSTOMERS_BKP table available which is a backup of the CUSTOMERS table. The following example deletes the records from the CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

```
DELETE FROM CUSTOMERS
 WHERE AGE IN
    (SELECT AGE FROM CUSTOMERS_BKP
     WHERE AGE >= 27 );
```

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00